Week 7: Lot Sizing, MRP Integration & Simulation

ISE 573 — Manufacturing Execution Systems

Manbir Sodhi

University of Rhode Island

Spring 2026

Week 7 Overview

The arc this week:

1. **Lot sizing heuristics** decide *how much* to order
2. **MRP + Python/SQLite** automates planning against a live database
3. **Demand simulator** stress-tests the system
4. **Discrete-Event Simulation (DES)** shows why deterministic plans fail
5. **MESA F4** explains where the data comes from

Transition from Week 6: You set up optimization models by hand in Excel. This week you *automate* and *stress-test* those plans with Python and SQL.

Learning Objectives

At the end of this week you will be able to:

1. Calculate EOQ/EPQ; apply Silver-Meal and Wagner-Whitin to dynamic demand
2. Formulate lot sizing as an optimization problem and solve it
3. Design and populate a SQLite factory database using Python
4. Implement MRP netting logic in Python against the database
5. Connect MRP to a demand simulator and observe nervousness
6. Explain DES fundamentals and identify appropriate simulation software
7. Describe MESA F4: data sources, capture methods, downstream feeds
8. Outline how this applies to your term project system design

Lot Sizing Is an Optimization Problem

Decision variable: Q = order quantity (how many units per order)

Objective: Minimize total cost per year

Constraints: $Q > 0$, demand must be satisfied, no stockouts

Formulation:

$$\min_{Q>0} \quad \mathrm{TC}(Q) = \underbrace{S \cdot \frac{D}{Q}}_{\text{setup cost}} + \underbrace{H \cdot \frac{Q}{2}}_{\text{holding cost}}$$

This is a **single-variable unconstrained optimization**: convex function, unique minimum. For dynamic demand over multiple periods, it becomes a **combinatorial optimization** — which periods to group into each order.

Economic Order Quantity (EOQ) — Definition

EOQ = Economic Order Quantity — the order size that minimizes the total of ordering cost and inventory holding cost under constant demand.

Assumptions:

- Demand is constant and known: D units per year
- Lead time is constant and known
- Entire order arrives at once (instantaneous replenishment)
- No stockouts permitted
- Costs (S, H) are stationary

First published by Ford W. Harris (1913). Still widely used because it is *robust* — the cost curve is flat near the optimum.

EOQ Derivation

$$\mathrm{TC}(Q) = S \cdot \frac{D}{Q} + H \cdot \frac{Q}{2}$$

Take derivative, set to zero:

$$\frac{d\,\mathrm{TC}}{dQ} = -\frac{SD}{Q^2} + \frac{H}{2} = 0 \quad \Longrightarrow \quad Q^2 = \frac{2SD}{H}$$

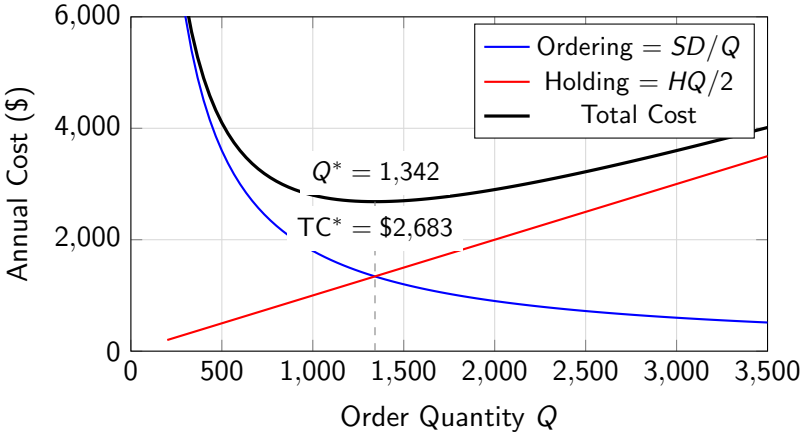
$$\boxed{Q^* = \sqrt{\frac{2SD}{H}}}$$

At the optimum: ordering cost = holding cost:

$$S \cdot \frac{D}{Q^*} = H \cdot \frac{Q^*}{2} = \sqrt{\frac{SDH}{2}}$$

Total cost at optimum: $\mathrm{TC}^* = \sqrt{2SDH}$

EOQ — Cost Curves



EOQ Worked Example

Problem: $D = 12{,}000$ brackets/year, $S = \$150$/order, $H = \$2$/unit/year.

Step 1: $Q^* = \sqrt{2 \times 150 \times 12{,}000/2} = \sqrt{1{,}800{,}000} = \mathbf{1{,}342}$

Step 2: Orders/year $= 12{,}000/1{,}342 = 8.94 \approx$ every 41 days

Step 3:

$$\begin{aligned} \text{Ordering} &= 150 \times 8.94 = \$1{,}341 \\ \text{Holding} &= 2 \times 671 = \$1{,}342 \\ \text{Total} &= \$2{,}683 \quad (\text{ordering} \approx \text{holding}\checkmark) \end{aligned}$$

Step 4 — Sensitivity: Demand doubles $\Rightarrow$ Q^* up $\sqrt{2} = 41\%$, not double. Holding $+50\% \Rightarrow$ Q^* down 18%. The flat cost curve makes EOQ *robust*.

Economic Production Quantity (EPQ) — Definition

EPQ = Economic Production Quantity — the production lot size that minimizes total cost when items are *produced* at a finite rate P while being *consumed* at rate D.

Unlike EOQ (entire order arrives at once), EPQ accounts for inventory building *gradually* during production. Inventory peaks at $Q(1 - D/P)$, not Q.

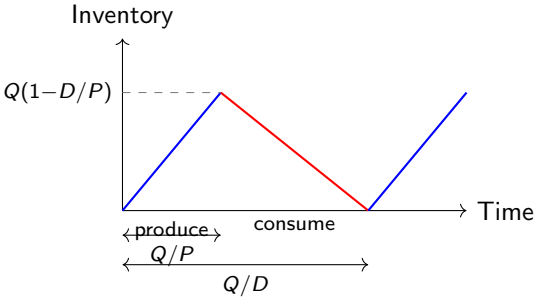
$$\text{Average inventory} = \frac{Q}{2}\left(1 - \frac{D}{P}\right)$$

$$\boxed{Q_p^* = \sqrt{\frac{2SD}{H\left(1 - \frac{D}{P}\right)}}}$$

As $P \to \infty$ (instantaneous), EPQ $\to$ EOQ.

EPQ — Inventory Profile and Cost Curves

Inventory profile:



Example: Same data, $P = 20{,}000/\text{yr}$.

$$Q_p^* = \sqrt{\frac{3{,}600{,}000}{2 \times 0.4}} = \mathbf{2{,}121}$$

EPQ > EOQ (2,121 vs 1,342) because average inventory is lower for any given Q.

Cost comparison:
EOQ: TC = \$2,683
EPQ: TC = \$2,530
Savings from finite production rate.

Quantity Discounts — Setup

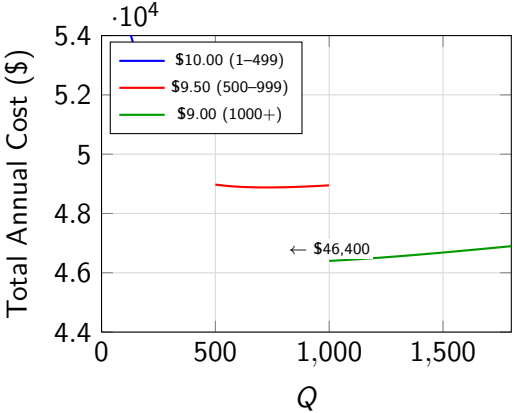
Suppliers offer price breaks:

Qty	Price	H
1–499	\$10.00	\$2.00
500–999	\$9.50	\$1.90
1000+	\$9.00	\$1.80

$D = 5{,}000/\text{yr}$, $S = \$100$.

Total cost now includes purchase:

$$\text{TC} = DP + S\frac{D}{Q} + H\frac{Q}{2}$$



Quantity Discounts — Solution

Step 1: EOQ at each price: 707, 726, 745.
Step 2: Feasibility — only 726 in its range (500–999). ✓
Step 3: Adjust: \$10 → Q=499, \$9 → Q=1,000.
Step 4: Compare:

Q	Price	Purchase\$	Order+Hold\$	**Total**
499	\$10.00	\$50,000	\$1,501	\$51,501
726	\$9.50	\$47,500	\$1,379	\$48,879
1,000	**\$9.00**	**\$45,000**	**\$1,400**	**\$46,400** ←

Lesson: Purchase cost often dominates. The EOQ at \$9 (745) isn't feasible, but ordering the *minimum* for the discount (1,000) still wins.

Dynamic Demand — Why EOQ Isn't Enough

EOQ assumes *constant* demand. Real demand varies:

Period	1	2	3	4	5	6
Demand	20	50	10	40	30	60

$S = \$200/\text{order}$, $H = \$2/\text{unit/period}$.

The optimization problem: Partition T periods into groups. Each group gets one order covering all its periods' demand. Minimize total setup + holding cost.

We'll solve with three methods: Lot-for-Lot, Silver-Meal, Wagner-Whitin.

Lot-for-Lot (L4L)

Order exactly what you need each period. Zero inventory.

Period	1	2	3	4	5	6
Demand	20	50	10	40	30	60
Order	20	50	10	40	30	60
End Inventory	0	0	0	0	0	0

- 6 orders $\times$ \$200 = \$1,200 ordering, \$0 holding
- **Total: \$1,200**
- Good when S is small or demand is highly uncertain

Silver-Meal Algorithm

Idea: Start a new order. Keep adding future periods while the *average cost per period* decreases. Stop when it increases.

Algorithm:

1. Start order at period t
2. Compute: avg cost/period if covering t through $t + k$
3. If average decreased $\Rightarrow$ increment k, continue
4. If average increased $\Rightarrow$ **stop**; order covers t to $t + k - 1$
5. Move to next uncovered period, repeat

Holding cost for period k demand carried from period j: $d_k \times H \times (k - j)$

Silver-Meal — Order 1 (Detailed)

Starting period 1. Evaluate extending coverage:

Cover	**Setup**	**Holding**	**Avg/Period**	
1 only	200	0	$200/1 = 200.00$	
1–2	200	$50 \times 2 \times 1 = 100$	$300/2 = 150.00$	$\downarrow$ continue
1–3	200	$100 + 10 \times 2 \times 2 = 140$	$340/3 = 113.33$	$\downarrow$ continue
1–4	200	$100+40+40\times2\times3 = 380$	$580/4 = 145.00$	$\uparrow$ **STOP**

Decision: Order covers periods 1–3. Quantity $= 20 + 50 + 10 =$ **80** in period 1.
Why stop? Adding period 4 (demand 40, carried 3 periods) adds so much holding cost that the average increases.

Silver-Meal — Orders 2, 3 and Summary

Order 2 (period 4): Cover 4 avg=200; 4–5 avg=130 $\downarrow$; 4–6 avg=167 $\uparrow$ Stop. Covers 4–5. Qty = 70 in period 4.

Order 3: Period 6 remains. Qty = 60 in period 6.

Period	1	2	3	4	5	6
Order	80	–	–	70	–	60
End Inventory	60	10	0	30	0	0

3 orders $\times$ \$200 = \$600. Holding = (60+10+30) $\times$ \$2 = \$200. **Total: \$800.** **Saved 33%** vs L4L.

Wagner-Whitin — The Optimization Formulation

This is the integer programming formulation for the dynamic lot sizing problem:

Sets: $t = 1, \ldots, T$ periods. **Data:** demand d_t, setup S, holding H.

Variables: $y_t \in \{0, 1\}$ (order placed in t?), $q_t \geq 0$ (quantity), $I_t \geq 0$ (ending inventory).

$$
\begin{aligned}
\min \quad & \sum_{t=1}^{T} (S \cdot y_t + H \cdot I_t) \\
\text{s.t.} \quad & I_{t-1} + q_t - d_t = I_t && \forall\, t \quad \text{(inventory balance)} \\
& q_t \leq M \cdot y_t && \forall\, t \quad \text{(link order to setup)} \\
& I_0 = 0,\ I_T = 0 \\
& y_t \in \{0, 1\},\ q_t \geq 0,\ I_t \geq 0
\end{aligned}
$$

Wagner-Whitin solves this *exactly* via dynamic programming in $O(T^2)$.

Wagner-Whitin — DP Solution

$f(t)$ = minimum cost through period t. $\quad f(0) = 0$.

$$f(t) = \min_{j=1,\ldots,t} \left\{ f(j-1) + S + H \sum_{k=j}^{t} (k-j) \cdot d_k \right\}$$

$f(t)$	**Best candidates**	**Result**
$f(1)$	j=1: $0 + 200 = 200$	**200**
$f(2)$	j=1: 300; j=2: 400	**300** via j=1
$f(3)$	j=1: 340; j=2: 420; j=3: 500	**340** via j=1
$f(4)$	j=1:580; j=2:580; j=3:580; j=4:540	**540** via j=4
$f(5)$	j=3:700; j=4:600; j=5:740	**600** via j=4
$f(6)$	j=4:840; j=5:860; j=6:800	**800** via j=6

Optimal: Order period 1 (qty 80), period 4 (qty 70), period 6 (qty 60). **TC** = \$800**.**

Wagner-Whitin — Python Implementation

```python
import math

def wagner_whitin(demand, S, H):
    T = len(demand)
    f = [0] + [math.inf] * T            # f[0..T]
    order_start = [0] * (T + 1)         # traceback

    for t in range(1, T + 1):
        for j in range(1, t + 1):
            hold = sum(H * (k - j) * demand[k - 1]
                       for k in range(j, t + 1))
            cost = f[j - 1] + S + hold
            if cost < f[t]:
                f[t] = cost
                order_start[t] = j

    # Traceback to get order plan
    plan = {}
    t = T
    while t > 0:
        j = order_start[t]
        plan[j] = sum(demand[j-1:t])
        t = j - 1
```

Method Comparison

Method	Orders	Order+Hold	Total	Optimal?
Lot-for-Lot	6	\$1,200+\$0	\$1,200	No
Silver-Meal	3	\$600+\$200	\$800	Heuristic
Wagner-Whitin	3	\$600+\$200	\$800	**Yes**

When to use which:

- **L4L:** Setup cost very small; or demand highly uncertain
- **Silver-Meal:** Fast, practical, usually within a few % of optimal
- **Wagner-Whitin:** Optimal but requires full demand horizon known. $O(T^2)$
- **MIP formulation:** For extensions (capacity, multiple items) — solve with PuLP/Gurobi

Bill of Materials — What It Is

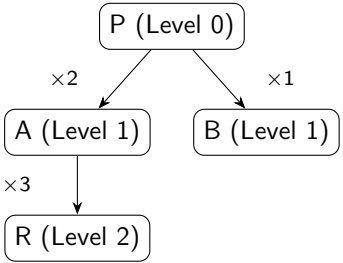
A **BOM** is a hierarchical list of all components, sub-assemblies, and raw materials needed to produce a finished product, including *quantities per parent*.

Key concepts:

Level 0	Finished product (what we sell)
Level 1	Direct components/sub-assemblies
Level 2+	Components of components, raw materials
qty_per	How many of the child per one parent
Low-level code	Deepest level at which an item appears (for MRP processing order)

BOMs can be **deep** (many levels, e.g., aircraft: 10+ levels) or **wide** (many components per level, e.g., electronics: 100s of parts).

BOM Example — Structure and Explosion



Reading the BOM:

1 unit of P requires:

- 2 units of Component A
- 1 unit of Component B

1 unit of A requires:

- 3 units of Raw Material R

Full explosion for 10 units of P:

- A: $10 \times 2 = 20$ units
- B: $10 \times 1 = 10$ units
- R: $20 \times 3 = 60$ units

BOM in the Database

```
import sqlite3

conn = sqlite3.connect('factory.db')
c = conn.cursor()

# BOM table: parent -> child with quantity
c.execute('''CREATE TABLE bom (
    parent_id TEXT, child_id TEXT, qty_per INTEGER,
    PRIMARY KEY (parent_id, child_id))''')

c.executemany("INSERT INTO bom VALUES (?,?,?)", [
    ('P', 'A', 2),   # 1 P needs 2 A
    ('P', 'B', 1),   # 1 P needs 1 B
    ('A', 'R', 3),   # 1 A needs 3 R
])

# Query: full explosion for item P
c.execute('''
    SELECT b1.parent_id, b1.child_id, b1.qty_per,
           b2.child_id AS grandchild, b1.qty_per * b2.qty_per AS total
    FROM bom b1
    LEFT JOIN bom b2 ON b1.child_id = b2.parent_id
    WHERE b1.parent_id = 'P'
```

Recursive BOM Explosion in Python

```python
def explode_bom(conn, item_id, qty_needed, level=0):
    """Recursively explode BOM and print requirements."""
    indent = "  " * level
    print(f"{indent}{item_id}: need {qty_needed}")

    cur = conn.cursor()
    cur.execute("SELECT child_id, qty_per FROM bom WHERE parent_id=?",
                (item_id,))
    children = cur.fetchall()

    for child_id, qty_per in children:
        child_qty = qty_needed * qty_per
        explode_bom(conn, child_id, child_qty, level + 1)

# Usage:
explode_bom(conn, 'P', 10)
# Output:
# P: need 10
#   A: need 20
#     R: need 60
#   B: need 10
```

This recursive function handles BOMs of *any depth* — the same code works for 3-level or

MRP Algorithm — Step by Step

Process items level by level (0 first, then 1, then 2, . . .)

For each item at each level:

1. **Get gross requirements**
   - Level 0: from MPS/demand table
   - Level 1+: from parent's planned releases $\times$ qty_per
2. **For each week:**
   - Projected on-hand = previous OH − gross requirement
   - If projected $<$ safety stock $\Rightarrow$ net requirement exists
   - Apply lot-sizing rule (L4L, FOQ, etc.) to get order quantity
   - Planned receipt = this week; planned release = receipt − lead time
   - If release week $< 1 \Rightarrow$ flag as **past-due**
3. **Planned releases become gross requirements** for children via BOM

Full Database Schema in Python

```python
import sqlite3

conn = sqlite3.connect('factory.db')
c = conn.cursor()

c.executescript('''
CREATE TABLE items (
    item_id TEXT PRIMARY KEY, name TEXT, level INTEGER,
    lead_time INTEGER, lot_rule TEXT, lot_size INTEGER,
    safety_stock INTEGER DEFAULT 0, on_hand INTEGER DEFAULT 0);

CREATE TABLE bom (
    parent_id TEXT, child_id TEXT, qty_per INTEGER,
    PRIMARY KEY (parent_id, child_id));

CREATE TABLE demand (
    item_id TEXT, week INTEGER, quantity INTEGER,
    PRIMARY KEY (item_id, week));

CREATE TABLE planned_orders (
    item_id TEXT, week INTEGER, quantity INTEGER,
    receipt_week INTEGER, status TEXT DEFAULT 'planned',
    PRIMARY KEY (item_id, week));
```

Loading the Sample Data

```
# Items: id, name, level, lead_time, lot_rule, lot_size, SS, OH
c.executemany("INSERT INTO items VALUES (?,?,?,?,?,?,?,?)", [
    ('P','Product P',    0, 1, 'L4L', None,  0, 10),
    ('A','Component A',  1, 2, 'FOQ',   50, 10, 25),
    ('B','Component B',  1, 1, 'FOQ',   30,  5, 15),
    ('R','Raw Material R',2, 3, 'FOQ',  200, 20, 100),
])

# BOM relationships
c.executemany("INSERT INTO bom VALUES (?,?,?)", [
    ('P','A',2), ('P','B',1), ('A','R',3)])

# MPS for Product P
mps = [(2,15),(3,20),(4,10),(5,25),(6,15),(7,20),(8,30)]
c.executemany("INSERT INTO demand VALUES ('P',?,?)", mps)

conn.commit()
```

MRP for Product P — Detailed Walkthrough

P: Level 0, L4L, LT= 1, SS= 0, on-hand= 10.

Wk 1: Gross= 0. OH $= 10 - 0 = 10 \geq 0$. No order.

Wk 2: Gross= 15. OH $= 10 - 15 = -5 < 0$. Net= 5. L4L $\Rightarrow$ order 5. Receipt wk 2, release wk $2 - 1 =$ **wk 1**. OH $= -5 + 5 = 0$.

Wk 3: Gross= 20. OH $= 0 - 20 = -20$. Order 20. Release **wk 2**. OH= 0.

Wk 4: Gross= 10. Order 10. Release **wk 3**.

Wk 5–8: Same pattern. Each week: order = demand, release = week−1.

Week	1	2	3	4	5	6	7	8
Gross	0	15	20	10	25	15	20	30
Plan. Release	**5**	**20**	**10**	**25**	**15**	**20**	**30**	**–**

Component A — Netting with FOQ and Safety Stock

A: Level 1, FOQ= 50, SS= 10, LT= 2, OH= 25.
Gross $= 2\times$ P releases: 10, 40, 20, 50, 30, 40, 60.

Wk 1: OH $= 25 - 10 = 15$. $15 \geq$ SS $= 10$ ✓. No order.

Wk 2: OH $= 15 - 40 = -25$. $-25 < 10$. Net $= 10 - (-25) = 35$. FOQ $\Rightarrow 50$.
OH $= -25 + 50 = 25$. Receipt wk 2 $\Rightarrow$ release wk $2 - 2 =$ **wk 0 (PAST DUE)**

Wk 3: $25 - 20 = 5 < 10$. FOQ= 50. OH= 55. Release **wk 1**.

Wk 4: $55 - 50 = 5 < 10$. FOQ= 50. OH= 55. Release **wk 2**.

Wk 5: $55 - 30 = 25 \geq 10$. No order.

Wk 6: $25 - 40 = -15 < 10$. FOQ= 50. OH= 35. Release **wk 4**.

Wk 7: $35 - 60 = -25 < 10$. FOQ= 50. OH= 25. Release **wk 5**.

Python MRP Engine — Core Loop

```python
import math

def run_mrp(conn):
    c = conn.cursor()
    c.execute("DELETE FROM planned_orders")

    for (level,) in c.execute(
            "SELECT DISTINCT level FROM items ORDER BY level"):
        for row in c.execute(
                "SELECT * FROM items WHERE level=?", (level,)):
            item = dict(zip([d[0] for d in c.description], row))
            mrp_item(conn, item)
    conn.commit()

def get_gross(conn, item):
    c = conn.cursor()
    if item['level'] == 0:
        return dict(c.execute(
            "SELECT week, quantity FROM demand WHERE item_id=?",
            (item['item_id'],)))
    gross = {}
    for pid, qp in c.execute(
            "SELECT parent_id, qty_per FROM bom WHERE child_id=?",
```

Python MRP Engine — Netting and Lot Sizing

```python
def mrp_item(conn, item):
    c = conn.cursor()
    gross = get_gross(conn, item)
    if not gross:
        return

    oh = item['on_hand']
    ss = item['safety_stock'] or 0
    lt = item['lead_time']

    for wk in sorted(gross):
        projected = oh - gross[wk]
        if projected < ss:
            net = ss - projected
            # Apply lot-sizing rule
            if item['lot_rule'] == 'FOQ' and item['lot_size']:
                qty = math.ceil(net / item['lot_size']) \
                      * item['lot_size']
            else:  # L4L
                qty = net
            oh = projected + qty
            rel = wk - lt
            status = 'past_due' if rel < 1 else 'planned'
```

What Is MRP Nervousness?

MRP nervousness = small changes in the Master Production Schedule cause *large, cascading* changes in planned orders at lower BOM levels.

In our example: A modest MPS of 15–30 units of P per week generated:

- A past-due order for Component A (week 0 release)
- Multiple past-due orders for Raw Material R (3-week lead time)
- Excess inventory from FOQ rounding (ordering 50 when we need 5)

Now imagine the MPS changes slightly next week. *All* of those planned orders shift — some appear, some disappear, quantities change. The shop floor can't keep up.

Causes of Nervousness

1. **Long lead times at lower levels**
   A 3-week lead time for R means a change in week 5 demand ripples back to a release in week 2. Re-plan next week and that release might move to week 1.

2. **Large fixed lot sizes (FOQ)**
   FOQ$=50$ when net need is $5 \Rightarrow$ we order $10\times$ what we need. Next period has surplus, following period doesn't. Creates *lumpy* demand downstream.

3. **Deep BOM structures**
   Each level amplifies changes. 3 levels with $2\times$ multiplier: a 1-unit MPS change becomes $2 \times 3 = 6$ units at Level 2.

4. **Frequent replanning**
   Re-running MRP daily means orders change daily. Shop floor receives contradictory signals.

Measuring Nervousness — The Experiment

Protocol:

1. Generate 12 weeks of stochastic demand with simulator
2. Run MRP $\Rightarrow$ record all planned orders (Plan A)
3. Generate *new* demand (same distribution, different random draws)
4. Run MRP again $\Rightarrow$ record planned orders (Plan B)
5. Compare Plan A vs Plan B:
   - How many orders *changed* (different week or quantity)?
   - How many orders were *added* or *deleted*?
   - What is the *total absolute change* in order quantities?

$$\textbf{Nervousness index} = \frac{\text{changed orders}}{\text{total orders in Plan A}} \times 100\%$$

Mitigating Nervousness

Strategy	How It Works	Trade-off
Time fences	Freeze orders inside fence	Less responsive
L4L at low levels	Reduces lumpiness	More orders
Reduce lead times	Less time for plans to shift	May cost more
Replan less often	Weekly not daily	Slower reaction
Safety stock	Buffer absorbs changes	Higher inventory
Dampening	Ignore changes $<$ threshold	May miss real changes

In the lab: You'll measure nervousness with FOQ vs L4L at different BOM levels and see which combination minimizes instability while keeping costs reasonable.

Demand Simulator in Python

```python
import random, math

def simulate_demand(conn, item_id, weeks, base, noise=0.3):
    """Generate stochastic demand with seasonal variation."""
    c = conn.cursor()
    c.execute("DELETE FROM demand WHERE item_id=?", (item_id,))

    for wk in range(1, weeks + 1):
        seasonal = 1.0 + 0.2 * math.sin(2 * math.pi * wk / 12)
        qty = max(0, round(
            base * seasonal * random.gauss(1.0, noise)))
        c.execute("INSERT INTO demand VALUES (?,?,?)",
                  (item_id, wk, qty))
    conn.commit()

# Generate demand, run MRP, save Plan A
simulate_demand(conn, 'P', 12, 20)
run_mrp(conn)
plan_a = dict(conn.execute(
    "SELECT item_id||week, quantity FROM planned_orders"))

# New demand, re-run, compare
simulate_demand(conn, 'P', 12, 20)
```

Measuring Nervousness in Python

```python
def nervousness_index(plan_a, plan_b):
    all_keys = set(plan_a) | set(plan_b)
    changed = 0
    for key in all_keys:
        a = plan_a.get(key, 0)
        b = plan_b.get(key, 0)
        if a != b:
            changed += 1
    total = max(len(plan_a), 1)
    return changed / total * 100

ni = nervousness_index(plan_a, plan_b)
print(f"Nervousness index: {ni:.1f}%")
# Typical result: 40-70% of orders change!
```

Students will run this experiment with:

- FOQ at all levels vs L4L at all levels
- Mixed strategies (FOQ for Level 0, L4L for Levels 1–2)
- Different safety stock levels

Why Simulate?

- MRP and lot sizing are **deterministic** — they assume known demand, lead times, and processing times
- Real factories have **variability**: breakdowns, demand fluctuations, quality issues, absenteeism
- The nervousness experiment showed demand variability breaking plans
- **Simulation** tests plans under realistic conditions *before* deployment

Key insight: Variability degrades performance *nonlinearly*. A machine at 90% utilization has $\sim 9\times$ the queue of one at 50%. You can't see this from a spreadsheet.

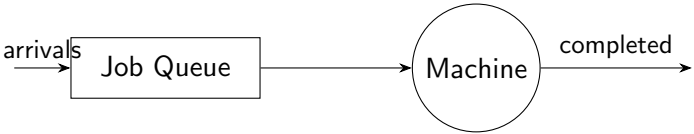
Discrete-Event Simulation (DES) — Core Concepts

DES models systems where state changes at discrete points in time.

Entities	Things that flow (jobs, parts, patients, packets)
Resources	Things that serve (machines, operators, beds, servers)
Events	Instantaneous state changes (arrival, start, finish, breakdown)
Queues	Where entities wait for busy resources
Clock	Jumps from event to event (nothing happens between)
Random variates	Interarrival times, processing times, failure rates

Next-event time advancement: Process the earliest event on the event list, update state, schedule new events, advance clock. Repeat until done.

DES Example — Single Machine Shop

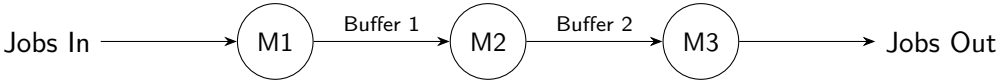


Events:

1. **Job arrives:** If machine idle ⇒ start processing. Else ⇒ join queue.
2. **Job finishes:** If queue not empty ⇒ start next job. Else ⇒ machine idle.
3. **Machine breaks down:** Current job paused, queue grows.
4. **Machine repaired:** Resume processing.

Outputs: Average wait time, queue length, utilization, throughput.

DES Example — Manufacturing Cell



Questions DES answers:

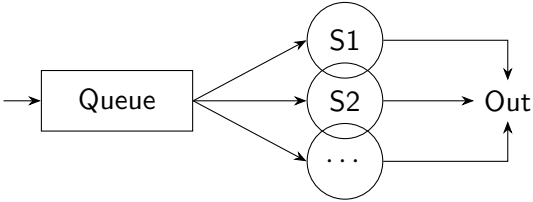
- What buffer sizes prevent *blocking* (upstream full) and *starving* (downstream empty)?
- Which machine is the *bottleneck*?
- What happens if M2 breaks down for 30 minutes?
- How many operators needed per shift?

These questions *cannot* be answered by deterministic models alone.

From M/M/1 to M/M/k — Adding Servers

M/M/1: one server, exponential arrivals and service. You understand the mechanics from the event loop.

M/M/k: k parallel servers sharing *one* queue. A job goes to the first idle server.



M/M/k analytical results:

$$\rho = \frac{\lambda}{k\mu} \qquad W_q = \frac{C(k, \lambda/\mu)}{k\mu - \lambda} \qquad L_q = \lambda W_q$$

$C(k, a)$ = Erlang-C formula. Key insight: *doubling servers reduces wait far more than halving arrival rate*. A finite buffer B additionally limits queue length, causing **blocking** when the queue is full.

MMkDES — Lab Experiments

Students run four experiments and record results in a table:

Config	k	B	ρ	**Avg wait**	**Blocked**
Baseline	1	∞	0.80	???	0
Two servers	2	∞	0.80	???	0
Finite buffer	1	5	0.80	???	???
High utilization	1	∞	0.95	???	0
Combined	2	5	0.95	???	???

Key questions to answer:

- Going from $k = 1$ to $k = 2$: how much does average wait decrease?
- At $\rho = 0.95$: does adding a second server *or* adding buffer space reduce wait more?
- What is the cost of blocking (lost jobs) vs. cost of a longer queue?

These questions have no clean closed-form answer once B is finite. Simulation is the only tool.

DES Philosophy — Building Our Own Tools

This course builds every simulation tool from scratch in Python.

Why not use commercial DES software?

- Black-box tools hide the mechanics
- Licenses are expensive outside academia
- You cannot extend or integrate them freely
- The event loop, queue logic, and statistics are the same regardless of the GUI

What we build instead:

Week 7 `MMkDES` — M/M/k with finite buffer

Week 8 `FlowshopDES` — 5-machine skip-routing shop

What students control (configure):

- Arrival rates, service rates
- Number of servers k
- Buffer capacity B
- Routing matrix (Week 8+)
- Failure rates (Week 9+)

What is provided (not coded):

- The event loop internals
- Statistics collection
- Report generation

Industry context: Arena, Plant Simulation, AnyLogic, and FlexSim all implement the

DES in Manufacturing — Real Applications

- **Toyota:** Uses Plant Simulation to design and validate production lines before physical construction. Saves months of trial-and-error.
- **Amazon fulfillment:** DES models warehouse picking, packing, and shipping to optimize layout and staffing.
- **Semiconductor fabs:** Extremely complex job shops (100s of machines, re-entrant flows). DES is the *only* way to predict cycle times.
- **Pharmaceutical:** Batch process simulation for campaign scheduling, cleaning validation, and capacity planning.
- **Aerospace:** Boeing and Airbus use DES for assembly line balancing and supply chain disruption analysis.

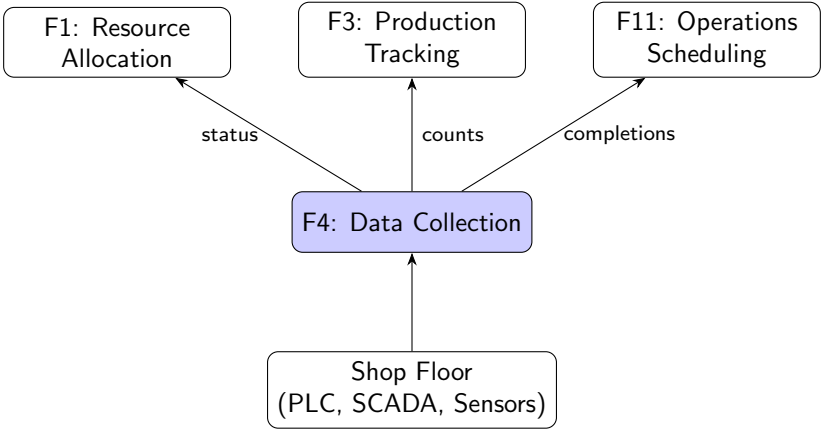
Common thread: **too complex for analytical models, too expensive to experiment on the real system.**

MESA F4: Data Collection & Acquisition

F4 captures the shop floor data that every other MES function depends on.

Data Type	Source	Frequency
Machine states (run/idle/down)	PLC / SCADA	Seconds
Cycle times	PLC counters	Per cycle
Piece counts (good/scrap)	Sensors / vision	Per piece
Quality measurements	CMM / gauges	Per sample
Operator actions	Badge / terminal	Per event
Material consumption	Barcode / RFID	Per transaction

F4 Feeds Other MES Functions



Without F4: F1 has no status data, F3 can't track, F11 can't react.
F4 is the foundation.

F4 and the Closed-Loop MRP

1. MRP generates planned orders (what *should* happen)
2. Shop floor executes (what *actually* happens)
3. **F4 captures actuals:** completions, scrap, consumption, delays
4. Database updates: on-hand inventory, WIP, machine states
5. MRP replans based on updated data

Without F4, MRP is **open-loop** — plans diverge from reality and nobody knows. With F4, MRP becomes **closed-loop** — deviations are detected and corrected.

Term Project Outline — What to Address

Your 2-page outline should cover:

1. **Database Schema:** What tables? How does your BOM map to the factory database?
2. **MRP / Planning Logic:** Which lot-sizing rules? Replanning frequency?
3. **Demand Interface:** Customer orders, forecasts, or MPS?
4. **Data Collection (F4):** What shop floor data? What sensors/methods?
5. **Simulation Plan:** What questions would DES answer for your system?

Week 7 Summary

Five takeaways:

1. **Lot sizing is an optimization problem** — EOQ, EPQ, Silver-Meal, Wagner-Whitin give you tools to solve it
2. **MRP is a mechanical algorithm** — implementable in ∼50 lines of Python + SQL
3. **Databases make it production-ready** — same logic, live data, audit trails
4. **Uncertainty breaks plans** — the demand simulator and nervousness index prove it
5. **F4 closes the loop** — without shop floor data, planning is open-loop guessing

Lab this week: Build your own MRP system with Python + SQLite; run M/M/k experiments with `MMkDES`.

Next week: Flowshop DES (5 machines, 10 part types), MESA F5, and Rough-Cut Capacity Planning.